Multi-Vehicle Navigation

Multi-Vehicle Navigation
1. Course Content
2. Preparation
3. Case Demonstration
4. Source Code Analysis

1. Course Content

- In multi-robot autonomous cooperative operation scenarios, multiple vehicles need to independently perform self-localization, costmap maintenance, and local dynamic obstacle avoidance planning under a shared global map. This course uses two vehicles (`robot1` and `robot2`) as an example. Through namespaces, the sensor topics, TF tree, and Nav2 navigation stack are isolated independently, demonstrating the complete workflow in which the two vehicles share one global map and navigate independently to designated goal points.
- This feature assigns different navigation goal points to the two vehicles in RViz2. Based on the unified global map and their respective AMCL Monte Carlo localization, the two vehicles autonomously plan globally optimal paths. During driving, they use LiDAR to detect unknown obstacles and the other vehicle in real time, avoid them, and finally reach their designated goals safely.

2. Preparation

- Configure the namespace and ROS DOMAIN ID
  - Refer to the content in the previous course 【Multi-Vehicle Control】 for the configuration operation.
- Pre-build the grid map of the environment
  - Use the primary vehicle (robot1) to build a grid map; for the mapping process, refer to the mapping course in the 【LiDAR】 chapter.

- Primary vehicle robot1 start

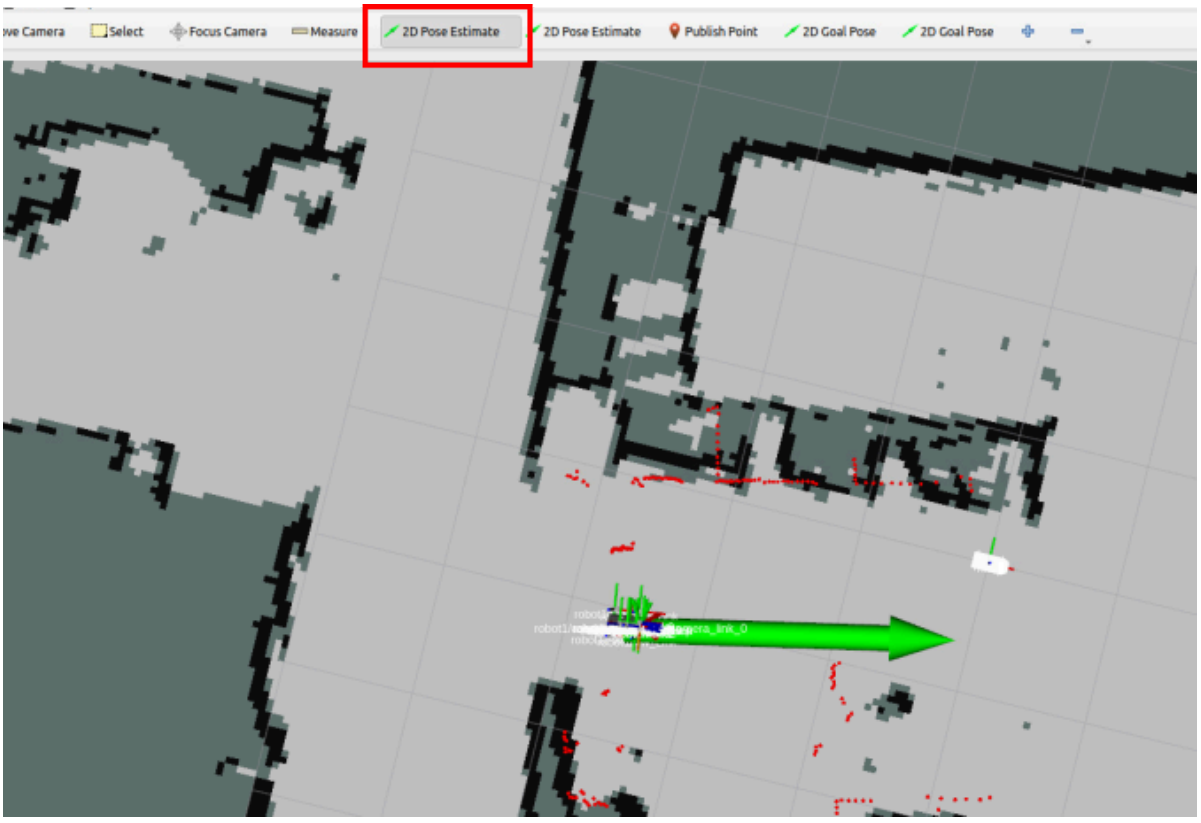
```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py demo:=navigation
use_map:=true use_rviz:=true
```

- Other follower vehicles start

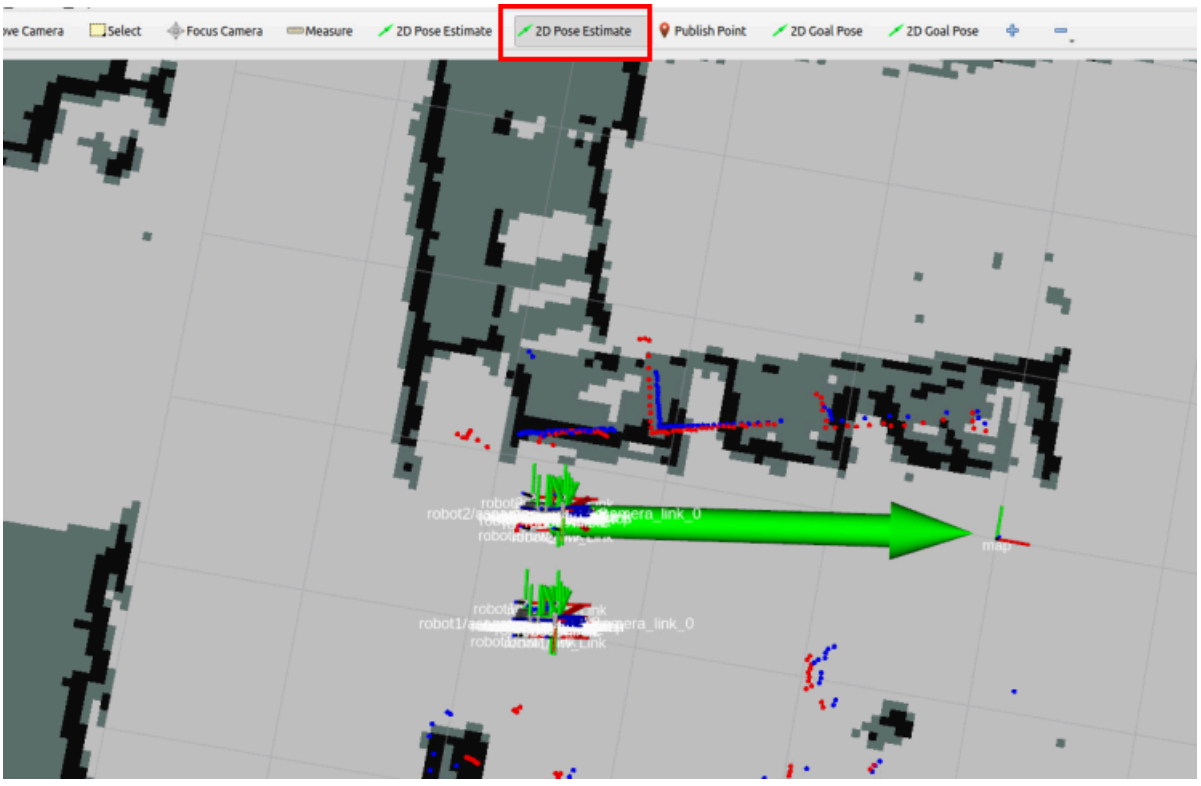
```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py demo:=navigation
```

- Observe the top toolbar of RViz, where two dedicated **【2D Pose Estimate】** tools are configured:
  - The first tool is remapped to `/robot1/initialpose`;

    - The second tool is remapped to `/robot2/initialpose`.
- Initialize the robot1 vehicle position: use the first 【2D Pose Estimate】 to initialize the position of vehicle 1; the LiDAR point cloud is red.

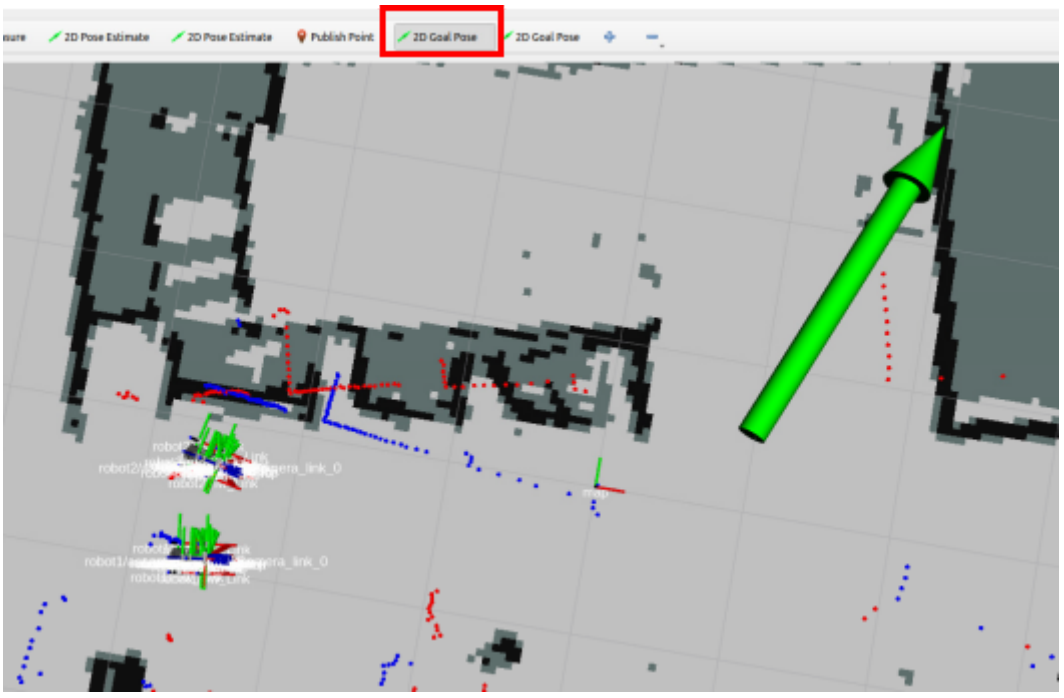


- Initialize the robot2 vehicle position: use the second 【2D Pose Estimate】 to initialize the position of vehicle 2; the LiDAR point cloud is blue.

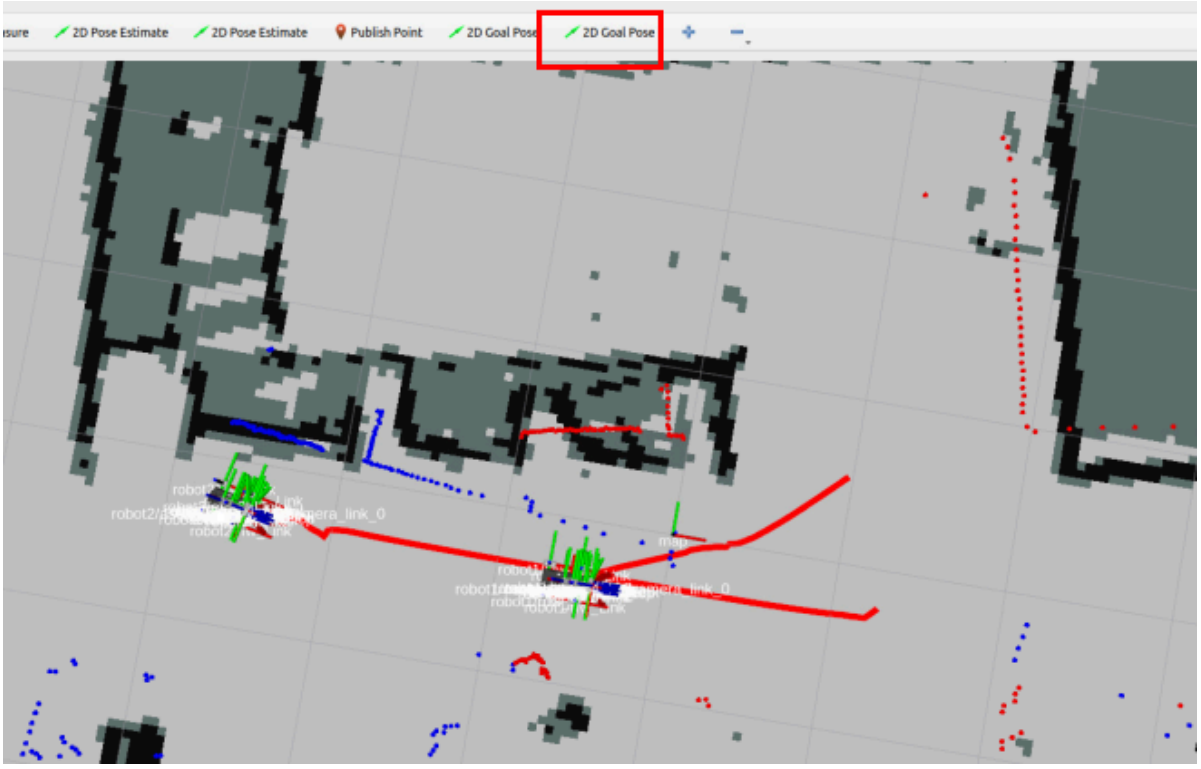


3. Case Demonstration

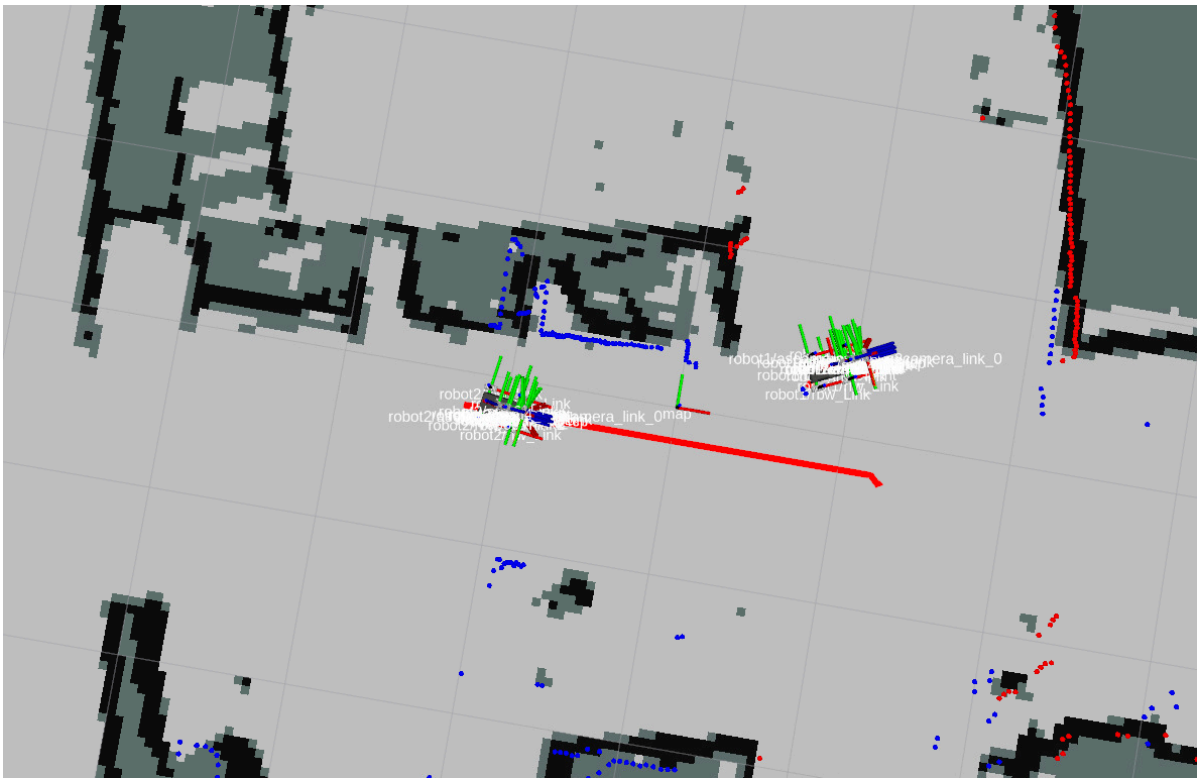
- Use the first 【2D Goal Pose】 to send a goal point to `/robot1/goal_pose`.

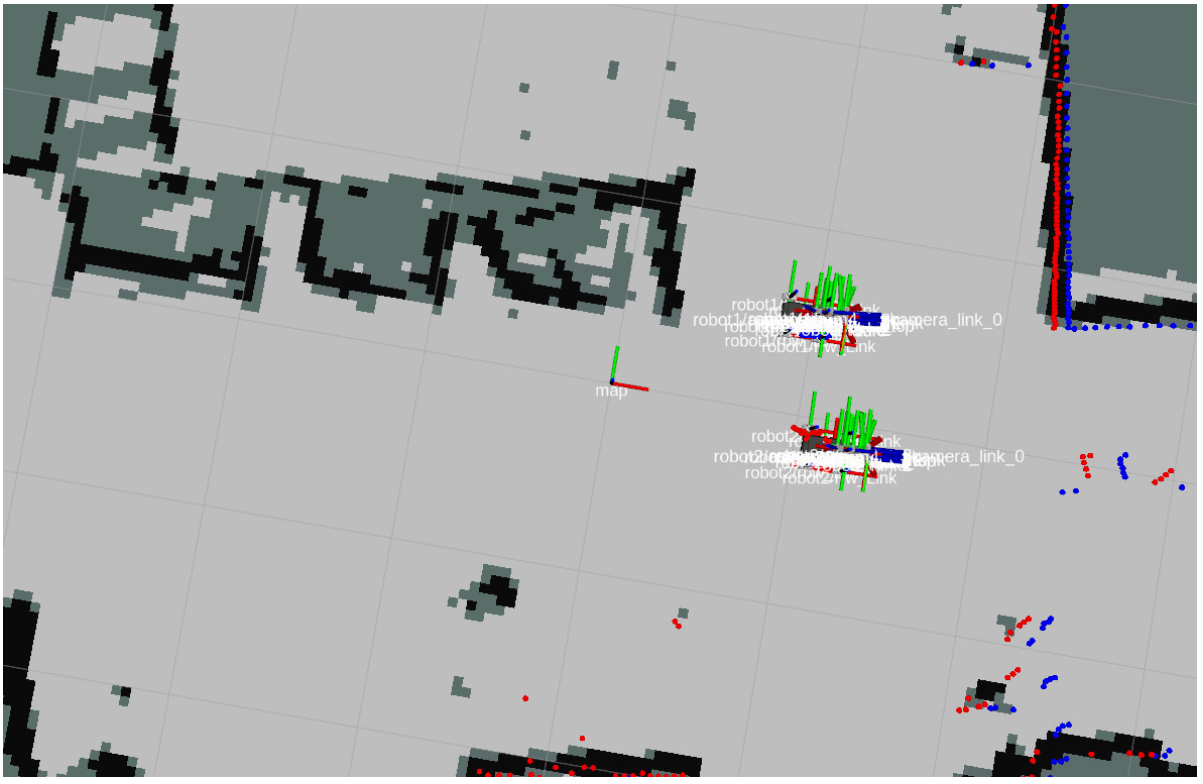


- Use the second 【2D Goal Pose】 to send a goal point to `/robot2/goal_pose`.



- The two vehicles set off along their respective global paths at the same time, avoiding obstacles in the map and each other.





4. Source Code Analysis

- Launch path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/multi_robot_demo
.launch.py
```

navigation_setup function description:

- `GetRosNamespace()`: Gets the namespace in which the current robot resides (such as `robot1`, `robot2`); the navigation nodes will run under this namespace.
- `map_file`: Concatenates the absolute path of the map file, reading the map file specified by the `map` parameter from the `~/map/` directory.
- `params_template`: Points to the multi-robot navigation parameter template file `nav2_multi_robot_params.yaml`, which contains the `<robot_ns>` placeholder.
- `use_map`: Controls whether to publish the map through the `use_map` launch parameter; in multi-robot scenarios, only the primary vehicle (robot1) publishes the map, and follower vehicles set it to `false`.
- `map_server` node: Loads and publishes the grid map; `yaml_filename` specifies the map YAML file path.
- `lifecycle_manager_map`: Manages the lifecycle of map_server; `autostart: True` means automatic startup.
- `if not ros_namespace: return actions`: If there is currently no namespace (i.e., non-multi-robot mode), only the map-related nodes are returned and then it returns directly.
- `RewrittenYaml` + `ReplaceString`: Replaces the `<robot_ns>` placeholder in the parameter template with the namespace of the current robot (such as `robot1`), achieving dynamic adaptation of the parameter file.
- `root_key=name`: Mounts the replaced YAML parameters under the root key of the corresponding namespace.
- `lifecycle_nodes`: Defines the list of component nodes in the Nav2 navigation stack that require lifecycle management.

Navigation node startup description:

- `amcl`: The Monte Carlo localization node, running under the robot namespace; `remappings=[('map', '/map')]` remaps the map topic to the globally shared `/map`, ensuring all robots use the same map.
- `controller_server`: The local path tracking controller, responsible for generating velocity commands.
- `planner_server`: The global path planner, planning optimal paths based on the global map.
- `behavior_server`: The behavior server, providing recovery behaviors (such as in-place rotation and backing up).
- `bt_navigator`: The behavior tree navigator, coordinating the decision logic during navigation.
- `waypoint_follower`: The waypoint follower, supporting sequential multi-point navigation.
- `lifecycle_manager_navigation`: Uniformly manages the lifecycle of the above 6 navigation components; `autostart: True` automatically starts all nodes.
- All navigation nodes share the same `params_file`, and namespace isolation is used to achieve independent localization, planning, and obstacle avoidance for each robot.

control_setup function description:

- `robot_names`: Parses the robot namespace list from the `robot_names` launch parameter, separated by commas, removing whitespace and slashes.
- `multi_robot_sync` node: Starts the multi-vehicle synchronization control node, distributing the global control topics to each robot's namespace.

generate_launch_description function description:

- `demo`: The launch mode selection parameter; can be `control` (multi-vehicle control), `navigation` (multi-vehicle navigation), or `None`.
- `map`: The shared map file name, defaulting to `map.yaml`, located in the `~/map/` directory.
- `use_rviz`: Whether to launch the RViz2 visualization interface, defaulting to `false`.
- `use_map`: Whether to publish the map; the primary vehicle sets it to `true`, and follower vehicles set it to `false`.
- `robot_names`: The robot namespace list, defaulting to `robot1,robot2`, used for topic synchronization in control mode.
- `multi_robot_base`: Includes the launch of `multi_robot_base.launch.py`, loading the robot base driver (chassis, LiDAR, depth camera, etc.).
- `control`: `GroupAction` + `IfCondition`, which starts the multi-vehicle control node only when `demo:=control`.
- `navigation`: `GroupAction` + `IfCondition`, which starts the navigation stack and the optional RViz2 only when `demo:=navigation`.

```
def navigation_setup(context, *args, **kwargs):
    # ros namespace from robot_info.json, navigation nodes run under this
namespace
    ros_namespace = GetRosNamespace()
    map_file = os.path.join(os.path.expanduser('~'), 'map',
                            LaunchConfiguration('map').perform(context))
    bringup_dir = get_package_share_directory('yahboomcar_bringup')
    params_template = os.path.join(bringup_dir, 'param',
'nav2_multi_robot_params.yaml')

    actions = []

    # Map server: controlled by use_map, only one robot publishes the map in
multi-robot scenarios
    use_map = LaunchConfiguration('use_map').perform(context).lower() == 'true'
    if use_map:
        actions += [
            Node(
                package='nav2_map_server',
                executable='map_server',
                name='map_server',
                output='screen',
                parameters=[{'yaml_filename': map_file, 'use_sim_time': False}],
            ),
            Node(
                package='nav2_lifecycle_manager',
                executable='lifecycle_manager',
                name='lifecycle_manager_map',
                output='screen',
```

```
                parameters=[{'autostart': True, 'node_names': ['map_server']}],
            ),
        ]

    if not ros_namespace:
        return actions

    name = ros_namespace
    params_file = ParameterFile(
        RewrittenYaml(
            source_file=ReplaceString(
                source_file=params_template,
                replacements={'<robot_ns>': name}),
            root_key=name,
            param_rewrites={},
            convert_types=True),
        allow_substs=True)

    lifecycle_nodes = ['amcl', 'controller_server', 'planner_server',
                       'behavior_server', 'bt_navigator', 'waypoint_follower']

    # NOTE: no /tf remapping here, all robots share the global TF tree
    actions += [
        Node(
            package='nav2_amcl',
            executable='amcl',
            name='amcl',
            namespace=name,
            output='screen',
            parameters=[params_file],
            # humble amcl has no map_topic param, remap to the shared global map
            remappings=[('map', '/map')],
        ),
        Node(
            package='nav2_controller',
            executable='controller_server',
            namespace=name,
            output='screen',
            parameters=[params_file],
        ),
        Node(
            package='nav2_planner',
            executable='planner_server',
            name='planner_server',
            namespace=name,
            output='screen',
            parameters=[params_file],
        ),
        Node(
            package='nav2_behaviors',
            executable='behavior_server',
            name='behavior_server',
            namespace=name,
            output='screen',
            parameters=[params_file],
        ),
```

```
        Node(
            package='nav2_bt_navigator',
            executable='bt_navigator',
            name='bt_navigator',
            namespace=name,
            output='screen',
            parameters=[params_file],
        ),
        Node(
            package='nav2_waypoint_follower',
            executable='waypoint_follower',
            name='waypoint_follower',
            namespace=name,
            output='screen',
            parameters=[params_file],
        ),
        Node(
            package='nav2_lifecycle_manager',
            executable='lifecycle_manager',
            name='lifecycle_manager_navigation',
            namespace=name,
            output='screen',
            parameters=[{'autostart': True, 'node_names': lifecycle_nodes}],
        ),
    ]
    return actions


def control_setup(context, *args, **kwargs):
    # robot namespaces that the root control topics are synced to, e.g.
"robot1,robot2"
    robot_names = [name.strip().strip('/') for name in
                   LaunchConfiguration('robot_names').perform(context).split(',')
                   if name.strip()]
    if not robot_names:
        return []
    return [
        Node(
            package='common_demo',
            executable='multi_robot_sync',
            name='multi_robot_sync',
            output='screen',
            parameters=[{'robot_names': robot_names}],
        ),
    ]


def generate_launch_description():

    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "demo",
            default_value="None",
            description="use  (control/navigation/None)",
        )
```

```
    )

    declared_arguments.append(
        DeclareLaunchArgument(
            'map',
            default_value='map.yaml',
            description='Map name under ~/map, shared by all robots'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'use_rviz',
            default_value='false',
            description='Whether to launch rviz2'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'use_map',
            default_value='true',
            description='Whether to publish the map (set false on non-primary
robots in multi-robot scenarios)'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'robot_names',
            default_value='robot1,robot2',
            description='Comma separated robot namespaces to sync control topics
to (used when demo:=control)'
        )
    )

    bringup_dir = get_package_share_directory('yahboomcar_bringup')
    multi_robot_base=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','multi_robot_bas
e.launch.py')))

    # multi robot control topic sync, only launched when demo is control
    control=GroupAction(
        condition=IfCondition(PythonExpression(["'", LaunchConfiguration('demo'),
"' == 'control'"])),
        actions=[
            OpaqueFunction(function=control_setup),
        ],
    )

    navigation=GroupAction(
        condition=IfCondition(PythonExpression(["'", LaunchConfiguration('demo'),
"' == 'navigation'"])),
        actions=[
            OpaqueFunction(function=navigation_setup),
            Node(
                package='rviz2',
                executable='rviz2',
```

```
                name='rviz',
                output='screen',
                arguments=['-d', os.path.join(bringup_dir, 'rviz',
'nav2_multi_robot.rviz')],
                condition=IfCondition(LaunchConfiguration('use_rviz')),
            ),
        ],
    )
```

- Multi-robot navigation parameter path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/nav2_multi_robot_
params.yaml
```

Method for distinguishing different robot parameters through namespaces:

- The parameter template file uses the `<robot_ns>` placeholder to mark the fields that need to be dynamically replaced. The launch file replaces `<robot_ns>` with the actual robot namespace (such as `robot1`, `robot2`) at runtime through `RewrittenYaml` + `ReplaceString`, so that each robot loads its own independent parameters.

Key parameter description:

- `base_frame_id: "<robot_ns>/base_footprint"`: The robot base coordinate frame; after replacement it becomes `robot1/base_footprint` or `robot2/base_footprint`, ensuring each robot uses its own TF coordinate frame.
- `odom_frame_id: "<robot_ns>/odom"`: The odometry coordinate frame; after replacement it becomes `robot1/odom` or `robot2/odom`, and each robot independently maintains its own odometry transform.
- `global_frame_id: "map"`: The global map coordinate frame; all robots share the same `map` coordinate frame, without a namespace prefix.

```yaml
amcl:
  ros__parameters:
    use_sim_time: False
    alpha1: 0.1
    alpha2: 0.1
    alpha3: 0.3
    alpha4: 0.3
    alpha5: 0.1   # lateral slip noise, only used by OmniMotionModel
    base_frame_id: "<robot_ns>/base_footprint"
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: "map"
    lambda_short: 0.1
```

```
    laser_likelihood_max_dist: 2.0
    laser_max_range: 16.0
    laser_min_range: 0.05
    laser_model_type: "likelihood_field"
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: "<robot_ns>/odom"
    pf_err: 0.05
    pf_z: 0.99   #0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: "nav2_amcl::OmniMotionModel"   # mecanum holonomic base,
matches MPPI motion_model: Omni
    save_pose_rate: 0.5
    sigma_hit: 0.2
    tf_broadcast: true
    transform_tolerance: 2.0
    update_min_a: 0.2
    update_min_d: 0.15
    z_hit: 0.5
    z_max: 0.05
    z_rand: 0.5
    z_short: 0.05
    scan_topic: scan

....
```